Table 6.1 The main modules available from the LangChain.js library (*continued*)

Module	Description
`langchain/chains`	Combines multiple components into reusable pipelines
`@langchain/core`	Core package that exposes many common utilities and functions
`@langchain/community`	Community packages
`langchain/vectorstores`	Various vector database store integrations
`langchain/storage`	Provides abstractions for storing data in memory or in databases
`langchain/document_loaders`	Provides helpers for loading documents from various sources
`@langchain/openai`, `@langchain/anthropic`	Provide LLM integration classes with various providers
`langchain/output_parsers`	Exposes classes that transform outputs from LLMs into a more suitable format

These modules work together to provide a comprehensive toolkit for building advanced LLM applications. As we progress, we'll work on how to use these modules effectively in our project.

6.1.1 Chaining calls with LangChain

Let's begin by exploring the concepts of chains and how to manage prompts with LangChain. Chains are a fundamental feature in LangChain that allow you to combine multiple operations into a single, reusable workflow. They are particularly useful for creating complex AI workflows that require multiple steps of processing, such as retrieving information, analyzing it, and generating a response.

Chains are constructed using specific functions that are available through the *runnables* interface. Runnables in LangChain provide a powerful way to create flexible and reusable workflows for processing data. They allow you to chain various operations, enabling complex interactions with LLMs and other tools.

HOW RUNNABLES WORK

A runnable is essentially a unit of work that can be invoked, batched, or streamed. Each runnable can take input, perform some processing, and produce output, which can then be passed to another runnable in a chain. Runnables use the LangChain Expression Language (LCEL), which is basically a declarative way of composing functions that are resistant to code changes and includes baked-in support for retries and fallbacks. Figure 6.1 illustrates a simple chain that consists of four main components, represented by rectangles.

The arrows between the shapes show the flow of data from one component to the next. Each component's output serves as the input for the subsequent component in the chain. The `toUpperCase` and `VowelCount` perform some number processing first before providing these numbers as a prompt to the LLM. The last chain is the `StringOutputParser`, which is a special function that processes the LLM's output and returns a formatted string.